



Week 9 Lecture 1

▼ Class	BSCCS2003
🕒 Created	@November 1, 2021 10:55 AM
🔗 Materials	
# Module #	48
▼ Type	Lecture
# Week #	9

Access Control

What is Access Control?

- Access → Being able to read/write/modify information
- Not all parts of an application is for public access
 - Personal, Financial Company, Grades
- Types of access →
 - read-only
 - read-write (CRUD)
 - modify but not create
 - ...

Examples

- Linux files →
 - owner, group → access your own files, cannot modify (or even read?) others
 - can be changed by the owner
 - "root" or "admin" or "superuser" has the power to change permissions
- Email →
 - you can read your own email
 - can forward an email to someone else → this is also access!

- E-commerce login →
 - shopping cart etc. visible only to the user
 - financial information (credit card, etc.) must be secure

Discretionary vs Mandatory

- **Discretionary**
 - you have control over who you share with
 - forwarding emails, changing file access modes etc. possible
- **Mandatory**
 - decisions made by centralized management → users cannot even share information without permission
 - Typically only in military or high security scenarios

Role based access control

- Access associated with "role" instead of "username"
- Example →
 - Head of the department has access to student records
 - What happens when HoD changes?
- Single user can have multiple roles
 - HoD, Teacher, Cultural advisor, sports club member, ...
- Hierarchies, Groups
 - HoD > Teacher > Student
 - HpD vs sports club member? → No hierarchy here

Attribute based access control

- Attribute
 - time of the day
 - some attribute of user (citizenship, age, ...)
- Can add extra capability over role-based

Policies vs Permissions

- Permissions
 - Static rules usually based on simple checks (does the user belong to a group)?
- Policies
 - More complex conditions possible
 - Combine multiple policies
 - Example →
 - Bank employees can view ledger entries
 - Ledger access only after 8AM on working days

Principle of least privilege

- Entity should have minimal access required to do the job
- Example → Linux file system
 - users can read system libraries but not write
 - some files like `/etc/shadow` are not even readable
 - you can install Python to local files using `"venv"` but not to system path
- Benefits

- better security → fewer people with access to sensitive files
- better stability → user cannot accidentally delete important files
- ease of deployment → can create template filesystems to copy

Privilege Escalation

- Change user or gain an attribute
 - `"sudo"` or `"su"`
- Usually, combined with explicit logging and extra safety measures
- Recommended
 - do **not** `sudo` unless absolutely necessary
 - never operate as `root` in a Linux/Unix environment unless absolutely necessary

Context: Web apps

- Admin dashboards, user access, etc.
- Gradebook example:
 - only admin should be able to add/delete/modify
 - users should have read permissions only to their own data

Enforcing

- Hardware level
 - Security key, hardware token for access, locked doors, etc.
- Operating Systems
 - filesystem access, memory segmentation
- Application level
 - DB server can restrict access to specific databases
- Web application
 - Controllers enforce restrictions
 - Decorators in Python used in frameworks like Flask